
Lossless LRU Caching Is a Strong Baseline for Cache-Aware MoE Routing

Pierre-Luc Bacon
Mila, Quebec AI Institute; Université de Montréal
PLACEHOLDER@umontreal.ca

Diego Calanzone
Mila, Quebec AI Institute
diego.calanzone@mila.quebec

Riyasat Ohib
Georgia Institute of Technology
PLACEHOLDER@gatech.edu

Abstract

Decoding a mixture-of-experts model is limited by memory movement, not arithmetic: when experts overflow GPU memory, every routing decision that misses becomes a CPU-to-GPU copy. Recent proposals learn to make this traffic temporally consistent – hold one expert set for many tokens, or reward a router directly for cache-friendly reuse – and have mostly been judged by proxies: switch rate, a simulated cache’s hit ratio. We serve instead, on real hardware with every copy paid on the decode path, and find that learned temporal structure does not reliably translate into measured throughput: a controller of our own design and a reward-fine-tuned router (CAR-MoE, Cache-Aware Routing for MoE, trained with GRPO against a simulated LRU hit-ratio reward) both lose to simple, overlooked baselines on the metric that matters – physically counted expert transfers – even where their own simulated instrument reports gains. One measured constant, the per-expert transfer time τ , predicts every policy’s decode cost within $\sim 2\%$, and the real headroom is not where these methods look: a clairvoyant eviction oracle moves $2.1\times$ fewer experts than LRU at zero quality cost, locating the profitable learning problem in eviction, not in termination or reward shaping. We extend this measurement to two additional backbones, Phi-tiny-MoE-instruct and OLMoE-1B-7B, evaluating CAR-MoE, an option-critic-style controller, and MELINOE under an independent, estimate-based protocol (compute plus a per-miss transfer penalty rather than a real copy), and find a compatible story: our own routing-diversity analysis shows the option-critic controller’s near-perfect simulated hit ratio comes from collapsing the router onto a handful of experts, at considerable cost to downstream task accuracy, while CAR-MoE keeps routing close to load-balanced. Read together with the

physical null result on OLMoE, the message for this line of work is the same one switch-rate proxies already taught: a metric not priced in moved bytes can improve while the served cost stands still.

1 Introduction

A sparse mixture-of-experts layer runs only a few experts per token, so most of its parameters sit idle in GPU memory; OLMoE-1B-7B uses 1B of its 7B parameters per token [1]. When the model does not fit, the standard remedy keeps expert weights in CPU memory and copies an expert to the GPU whenever a token’s router selects one that is not already resident, evicting whichever resident expert has gone unused longest [2]; the same design is entering mainline serving (vLLM pull request #37190, benchmarked on this very model). This LRU caching is *lossless*: the router always runs the experts it chose, so the model computes exactly what it would with everything resident. Its price is traffic – consecutive tokens select overlapping but not identical experts, and decoding waits on the resulting stream of copies.

Two lines of recent work try to cut that stream by changing what the router does rather than how eviction works. One draws on the option-critic framework [3] and extends MoE routing temporally: hold an expert set as a macro-action until a learned termination function decides to switch [4]. The other treats cache-friendliness as a reward and fine-tunes the router directly with policy gradients, using a simulated LRU cache’s hit ratio as the signal – the approach we call CAR-MoE (Cache-Aware Routing for MoE) in this paper, trained with GRPO [7, 8] rather than a bespoke auxiliary loss. Both have so far been judged by proxies internal to their own instrumentation: switch rate, or the simulated cache’s own hit-ratio readout. Whether either buys any *physical* serving time over the lossless LRU baseline that actually moves weights has not been directly measured until now.

We close that gap two ways. First, a serving harness keeps half of a frozen OLMoE-1B-7B’s expert weights in host memory only; every change to residency is a real, synchronous, timed copy, and quality and traffic are measured in the same run. The verdict is that on PCIe-4-class hardware, lossless per-token fetching beats every policy that holds a set fixed, and the same harness, applied to a released CAR-MoE checkpoint, finds its simulated hit-ratio gain does not reduce physically counted transfers at all. Second, we broaden the comparison to two more backbones (Phi-tiny-MoE-instruct, OLMoE-1B-7B) and three objectives (CAR-MoE, an option-critic-style controller, MELINOE [6]) under an estimate-based protocol – compute time plus a per-miss transfer penalty, calibrated against each backbone’s own base model – paired with a routing-diversity analysis that asks not just whether hit ratio improved, but how: by genuine temporal reuse of a load-balanced expert population, or by routing collapse onto a fixed handful of experts. The two measurement regimes disagree on whether CAR-MoE’s simulated gain is real, and agree on where the actual one-shot exploitable structure sits: eviction, not termination, and diversity-preserving reuse, not collapse.

2 Measurement Setup

Physical harness (OLMoE-1B-7B, one backbone). OLMoE-1B-7B, frozen. Eight of sixteen MoE layers (the least dwell-sensitive under a per-layer probe) are restricted: their 64 experts live only in pinned host memory, served from a per-layer GPU pool of capacity $C = 32$. Decode is teacher-forced at batch 1 over sixteen held-out sequences; quality is next-token NLL, throughput is measured with paired device events, and steady-state numbers exclude each sequence’s initial pool fill. A serving policy’s cost per token is

$$\text{ms/token} = c + \rho \cdot \tau, \quad (1)$$

where c is compute time, ρ the number of experts moved per token, and τ the measured time to move one – a hardware constant that spans $5\times$ across GPUs from PCIe 5 to PCIe 3. Calibrated by one measured τ , Equation 1 reproduces measured decode time within $\sim 2\%$ when copies are physically slowed $10\times$ and $50\times$, so a single startup measurement of τ identifies the best policy at any tier.

Estimate-based cross-backbone protocol (Phi-tiny-MoE-instruct, OLMoE-1B-7B). We fine-tune both backbones with LoRA adapters (Phi: 16 experts, top-2; OLMoE: 64 experts, top-8) on the Nemotron Post-

Training Dataset v2 [10], comparing base, supervised fine-tuning (SFT) alone, CAR-MoE (a simulated fixed-capacity LRU working-set hit ratio as the GRPO reward at one middle layer), an option-critic-style controller [3, 4], and MELINOE. Unlike the physical harness above, throughput here is *not* a real copy: it is the base model’s own measured compute pass plus each objective’s own working-set miss count times a benchmarked per-expert disk-load constant, so an adapter that touches the expert projections directly is not penalized twice for its own inference-time cost, but the copy itself is never actually performed. We also compute the normalized routing entropy (ECI, $H(p)/\log(E)$, 1.0 = load-balanced, near 0 = collapsed onto a few experts) to distinguish genuine temporal reuse from routing collapse – a distinction the physical harness’s transfer count alone cannot make. Every score is averaged over two independent seeds where both were available at submission time.

3 Results

3.1 The measured frontier holds the lossless baseline

At the harness’s native τ , no policy that holds an expert set fixed beats exact LRU without paying a quality cost of at least $+0.067$ nats: on PCIe-4-class hardware, temporal extension does not pay. This is not a fixed verdict – what carries across hardware tiers is the transfer rate ρ , not the throughput, and Equation 1 converts any (ρ, c) pair to a throughput at any τ . At $50\times$ slowed copies, the ordering flips: low-traffic heuristics and the most switch-averse learned controller become the fastest restricted policies. Selecting a policy is a lookup, not a fixed preference: measure τ once, fix a quality tolerance, read the winner off Equation 1.

3.2 The Belady bound: the headroom is in eviction

Replaying the recorded per-token expert demand through a clairvoyant oracle that always evicts the expert needed furthest in the future [9] – certified within 4% of the provable optimum – yields exact serving (zero quality loss) at 4.6 transfers per token against LRU’s 9.6: a $2.1\times$ reduction for free. Holding a set fixed is therefore a quality-priced approximation of what better eviction gives away; the profitable learning problem is predicting near-future expert demand, not deciding when to stop holding a set. Sixteen tokens of foresight recover 96% of the oracle’s advantage, and a causally-trained scorer given only access history rediscovers a frequency heuristic – the useful temporal structure is already available losslessly, with a short horizon.

3.3 Learned approaches under physical and estimated measurement

The physical verdict on CAR-MoE. The same harness, applied to a CAR-MoE checkpoint (OLMoE-1B-7B fine-tuned with GRPO against a 16-slot simulated LRU hit-ratio reward at one middle layer, with a prompt-conditioned variant selecting the budget via a `[CACHE_SIZE=c]` tag), finds no reduction in physically counted transfers relative to the untuned base: 32,050 steady transfers against the base’s 31,915, and 10,963 against 10,954 even at the exact layer and capacity CAR-MoE was trained on. The conditioning tag makes traffic strictly worse. By its own instrument, training worked – the simulated hit rate at the trained layer improved – but the counter serving real bytes did not move. This is the same failure mode as the switch-rate proxy: a metric internal to the training-time simulator, not priced in moved bytes, can improve while nothing served changes.

The estimate-based cross-backbone verdict. Table 1 reports our own protocol’s downstream task deltas, working-set efficiency, and routing diversity across both backbones. CAR-MoE is competitive with or better than the base model on most tasks and improves hit ratio without collapsing the router (ECI stays within 0.01–0.02 of the base on both backbones); combined with the supervised loss (Phi-tiny-MoE only, so far), it raises hit ratio further while recovering the reward-only variant’s estimated throughput cost, which we read as evidence the reward combines readily with supervised learning rather than requiring extended justification on its own. The option-critic controller instead reaches a near-perfect simulated hit ratio by collapsing routing onto a handful of experts (ECI falls from 0.89 to 0.28 on Phi-tiny-MoE), at a considerable cost to downstream accuracy – exactly the kind of instrument-internal gain the physical harness would need to confirm independently, and which §3.3’s OLMoE re-measurement suggests should not be taken on faith.

Table 1: Estimate-based protocol, both backbones, at each backbone’s trained working-set capacity (Phi: $c = 4$, top-2, 16 experts; OLMoE: $c = 16$, top-8, 64 experts). Base row is absolute; other rows are Δ from their own backbone’s base. Avg Δ : mean of the five task deltas. Misses: mean working-set misses per sequence (lower better). ECI: normalized routing entropy (1.0 = load-balanced, 0 = collapsed). CAR-MoE rows highlighted; OLMoE CAR-MoE (+SFT) still running (–). Recall from §3.3 that a physical re-measurement of an OLMoE CAR-MoE checkpoint found no reduction in actual transferred bytes despite a comparable simulated hit-ratio gain – these numbers should be read as an internal, proxy-based comparison across objectives, not as a claim about served throughput.

Objective	MMLU		MMMLU		lm-eval-harness score (Δ from base)				MATH		Avg Δ		working-set efficiency				ECI	
	Phi OL		Phi OL		GSM8K		HumanEval		Phi OL		Phi OL		Hit ratio		Misses		Phi OL	
	Phi	OL	Phi	OL	Phi	OL	Phi	OL	Phi	OL	Phi	OL	Phi	OL	Phi	OL	Phi	OL
Base (untuned)	0.62	0.56	0.34	0.38	0.62	0.68	0.03	0.28	0.09	0.06	–	–	0.57	0.69	649.8	1874.2	0.89	0.90
SFT	+0.00	–0.02	–0.02	–0.01	–0.02	–0.09	+0.01	–0.13	–0.00	–0.02	–0.01	–0.05	0.56	0.68	742.1	1943.9	0.89	0.90
Option-critic	–0.03	–0.04	–0.02	–0.07	–0.11	–0.26	–0.02	–0.14	–0.08	–0.05	–0.05	–0.11	1.00	0.68	0.1	2012.7	0.28	0.89
MELINOE	–0.00	–0.07	–0.01	–0.06	–0.12	–0.31	–0.02	–0.20	–0.07	–0.05	–0.05	–0.14	0.88	0.76	237.7	1541.9	0.67	0.84
CAR-MoE	–0.01	–0.00	+0.00	–0.01	–0.05	–0.01	+0.00	+0.05	–0.01	+0.00	–0.01	+0.01	0.62	0.76	435.9	1004.2	0.89	0.90
CAR-MoE (+SFT)	+0.00	–	+0.00	–	–0.03	–	+0.00	–	–0.01	–	–0.01	–	0.78	–	421.4	–	0.71	–

Reading the two protocols together. The physical harness and the estimate-based protocol disagree on whether CAR-MoE’s hit-ratio gain is real, because they measure different things: the harness counts actual copies on one backbone at one trained configuration; our protocol estimates a throughput number from compute time and a miss count that is never physically moved, across two backbones and several objectives. Where they agree is more important than where they differ: neither treats a simulated cache’s own hit-ratio readout as sufficient evidence of anything served, and the routing-diversity analysis available only in our protocol shows that a high simulated hit ratio can come from two very different mechanisms – genuine, load-balanced temporal reuse (CAR-MoE) or router collapse onto a fixed subset (option-critic) – that a transfer count or a hit ratio alone cannot distinguish, but that downstream task accuracy and a physical re-measurement both would.

4 Related Work

Shen and Henderson [4] introduce the temporally extended mechanism judged by switch rate on a resident model, with a deliberation cost against degenerate options [3, 5]; the missing number was the exchange rate from switches to seconds, since their released implementation moves no weights and the memory saving is arithmetic rather than physical. Systems that do move expert weights share the design measured here – experts in host memory, a fixed GPU cache, misses paid over the link – and differ only in a hand-picked eviction rule, recency [2] or decayed frequency; none prices its rule against a measured hardware cost, and none learns eviction. MELINOE [6] instead keeps the routing architecture and adds a cache-simulation loss with a rank-matching regularizer against a frozen reference; we treat it here as a third point in the same proxy-judged family as the option-critic controller and CAR-MoE.

5 Limitations and Conclusion

The physical verdict is measured on one frozen 1B-active model, one text distribution, and batch-1 teacher-forced decode, a regime batching dissolves; the cross-backbone protocol substitutes an estimated cost model for a real copy, so its throughput numbers are a proxy in exactly the sense the physical harness warns against, and should be read as an internal comparison across objectives rather than a claim about served performance. Priced in real bytes, cache-aware MoE routing is a hardware question: one measured constant and each policy’s transfer rate pick the winner at every tier, fast links keep the lossless baseline, and proxy-aimed learning – switch rate, a simulated cache’s own hit ratio – can buy nothing the counter sees. Where CAR-MoE differs qualitatively from the option-critic controller is not in whether its simulated gain is physically confirmed (it was not, on the one checkpoint physically re-measured), but in *how* it reaches that gain: without collapsing the router’s routing diversity, and combining readily with supervised fine-tuning. The eviction oracle’s factor-of-two headroom, not reward-shaped or architecturally hard-coded temporal holding, remains where we think the profitable learning problem sits.

References

- [1] Muennighoff, N., Soldaini, L., Groeneveld, D., Lo, K., Morrison, J., Min, S., Shi, W., Walsh, P., Tafjord, O., Lambert, N., et al. OLMoE: Open Mixture-of-Experts Language Models. *arXiv preprint arXiv:2409.02060*, 2024.
- [2] Eliseev, A. and Mazur, D. Fast Inference of Mixture-of-Experts Language Models with Offloading. *arXiv preprint arXiv:2312.17238*, 2023.
- [3] Bacon, P.-L., Harb, J., and Precup, D. The Option-Critic Architecture. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2017.
- [4] Shen, Y. and Henderson, P. Temporally Extended Mixture-of-Experts via Option-Critic. *arXiv preprint arXiv:2604.20156*, 2026.
- [5] Harb, J., Bacon, P.-L., Klissarov, M., and Precup, D. When Waiting is not an Option: Learning Options with a Deliberation Cost. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.
- [6] Raje, A., Nayak, A., and Joshi, A. MELINOE: Fine-Tuning Mixture-of-Experts Language Models for Cache-Friendly Offloaded Inference. *arXiv preprint arXiv:2602.11192*, 2026.
- [7] Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y. K., Wu, Y., and Guo, D. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*, 2024.
- [8] Yu, Q., Zhang, Z., Zhu, R., Yuan, Y., Zuo, X., Yue, Y., Fan, T., Liu, G., Liu, L., Liu, X., et al. DAPO: An Open-Source LLM Reinforcement Learning System at Scale. *arXiv preprint arXiv:2503.14476*, 2025.
- [9] Belady, L. A. A Study of Replacement Algorithms for a Virtual-Storage Computer. *IBM Systems Journal*, 5(2):78–101, 1966.
- [10] NVIDIA. Nemotron Post-Training Dataset v2. *Hugging Face Dataset Card*, 2025.